MSO2520 Machine Learning: Technical Report



Middlesex University

20.04.2026

Objective:

The objective of this Coursework Project is to analyse and model the relationship between measurements and PCR outcomes. This project requires the clear demonstration and mathematical usage of:

- Linear Algebra
- Probability and Maximum Likelihood
- Optimisation
- Geometric Classification

The components required to complete the objective was the following:

1) Exploratory Linear Algebra

Covariance Analysis, Eigenvalues/Eigenvectors or rank analysis, Brief Geometric Interpretation.

2) Logistic Regression

Mathematical Formulation, Log-Likelihood Derivation, Optimisation Via Gradient Descent.

3) K-Nearest Neighbours (KNN)

Distance-Based Classification, Discussion of Metric Choice.

4) Support Vector Machine (SVM)

Margin Interpretation, Explanation of Optimisation Objective, Use of Linear SVM.

5) Comparative Analysis

Compare the Above Three Approaches, Interpret differences mathematically.

Report:

1. Introduction:

This report's goal is to analyse the dataset provided and model any potential relationship between measurements and PCR outcomes to attempt to display a correlation that may exist between Data and Outcome.

Our data contains the Haematological, Biochemical, and Immunological data from patients who underwent the PCR test, and using such information to predict whether said patient will test Positive or Negative for a particular illness. This dataset is potentially linked to COVID-19 PCR testing (not explicitly stated, only hinted), which, the outcomes of said model, if deemed reliable enough, may result in it not only being used more widely, but also having the model altered to test for different illnesses (Such as HPV, HIV, Tuberculosis, or any other pathogenic illness) in the future.

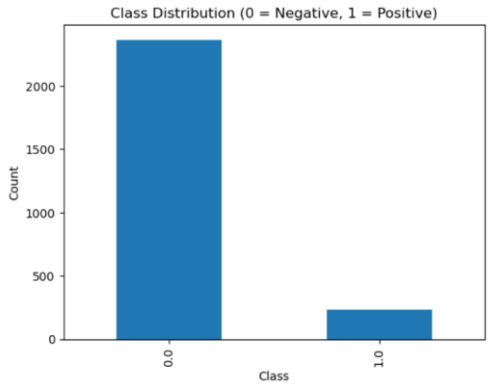
1.1 Brief Plan:

The plan of action with this report is to briefly explain the mathematics behind what I've used, the outcome at which said maths would achieve (as shown through the accompanying .ipynb file), and what each result means for the data.

1.2 Data Preparation:

Our data was cleaned of any potential NA entries, which did not exist in the given dataset analysed.

Our data is severely unbalanced with a 91-9 ratio split (91% being negative "0", 9% being positive "1") and as such, during our analysis of this data, we reweight our data to try and equally represent both sides.



The provided Bar Graph's purpose is to highlight just how great the difference between both classes, and to justify why equal weighting is needed.

2. Data Analysis:

During the preprocessing stage, we produced both a training and test set, our training set being used to help the model learn the patterns present with the data, whilst our test set will be used to see how effective and accurate this model is in predicting a result.

In mathematical terms, we started with:

$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$: Calculating the Mean of Our Data

$\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2}$: Calculating the Standard Deviation of Our Data

Then standardised our data:

$X'_{train} = \frac{X_{train} - \mu}{\sigma}$

$X'_{test} = \frac{X_{test} - \mu}{\sigma}$

This standardisation is required due to the number of different parameters present; this'll prevent one feature unjustifiably affecting another. In this case, we have 37 different parameters, all of them have their own range of acceptability that, often, aren't compatible with each other.

2.1 Exploratory Linear Algebra:

To start off with our Exploratory Linear Algebra, we calculated the covariance matrix of this data, using the following formula:

$$C = \frac{X_c^T X_c}{n - 1}$$

and

$$X_c = X_{train} - \overline{X}$$

In this situation, C represents our Covariance Matrix, X_c represents the centred version of our X_{train}, and $\overline{X} = \frac{1}{n}\sum_{i=1}^{n} x_i$, and n is our number of patients within our training data, in this case, n = 2077.

We divided by n – 1 instead of just n to ensure that our sample covariance using our training data, is unbiased, and represents the wider populations true covariance.

Our Covariance Matrix of $C \in \mathbb{R}^{37\times37}$, as the data has 37 different parameters, each row and column corresponds to some feature, every pair of parameters is represented and has a covariance value recorded in the matrix itself.

Now, with the Covariance Matrix stated, we can calculate both eigenvalues and eigenvectors of the Covariance matrix, this lets us find the Rank of our Covariance Matrix, determine how complex it is, and how much each piece of data contributes variance wise.
From our computations in calculating the Rank of this 37x37 Matrix, we get a value of 37, we have a full rank matrix, meaning that all values are linearly independent of one another, no parameter will explain another, which also lets us infer that we need each parameter to properly make an accurate prediction.

By using our eigenvalues we had computed within our code, alongside the following formula;

$$P_k = \frac{\lambda_k}{\sum_{i=1}^{37} \lambda_i}$$

Letting P_k be our explained variance ratio.

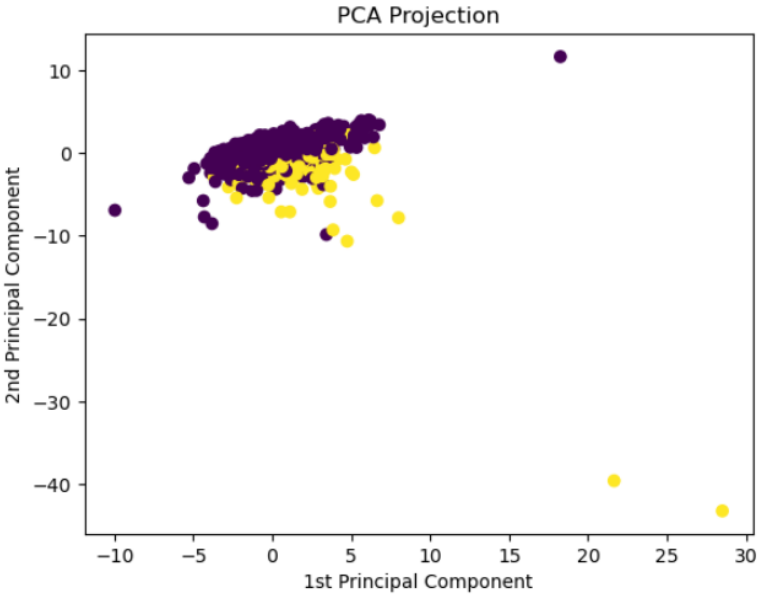
We can state that, only 27 out of our 37 Parameters were required for 95% Variance within our data, which represents how its 27 parameters which contribute the vast majority to our predictions, whilst the last 10 only contribute 5% to it.

Now, taking our eigenvectors that we had calculated as well, we can use python to plot a Principal Component Analysis, with our X-Axis representing the 1st Principal Component, and our Y-Axis representing the 2nd Principal Component.

Formula used before plotting:

$$X_2 = X_c W_2$$

Which projects our data only in 2 Dimensions for our PCA Plot.



This plot clearly shows a significant overlap between both the Positive and Negative classes, it cannot be easily separated with a single line, and as such, is not linearly separable in 2 dimensions, requiring the usage of other classification models, such as K-Nearest Neighbours, which can help decide a separation boundary at higher dimensions.

2.2 Logistic Regression:

Logistic regression [1] estimates the parameters of a logistic model (the coefficients in the linear or non-linear combinations). It's a form of classification algorithm which will predict some outcome.

The model that we are using, named "Training Logistic", is a type of regression model which uses our established parameters for each patient in the training data set for its predictions. To form our model/formula, we must also define multiple parts of it beforehand.

First, due to how severely unbalanced this data is towards the negative side, we must find a multiplier which can be applied to the positive side to equally weight them.

By summing both the negative and positive results and then dividing the number of negatives in our data by the positives, we get a positive weight of $\frac{1891}{186}$, which will be applied in our logistic regression to the positive values, counteracting our unbalanced data's weighting.

Regression Model:

$z = Xw$ (matrix multiplication)

$$p = \sigma(z) = \sigma(Xw)$$

$$weights = \frac{1891}{186}$$

$$weights_i = \begin{Bmatrix} \frac{1891}{186} & if\ y_i = 1 \\ 1 & if\ y_i = 0 \end{Bmatrix}$$

Defining a log likelihood function:

In Binary Outcomes:

If y = 1: probability is "p" = $\sigma(Xw)$

If y = 0: probability is "1-p" = $1 - \sigma(Xw)$

In a single formula, we can represent both situations as

$$P(y: x, w) = p^y \cdot (1-p)^{(1-y)}$$

Since, When:

y = 1: $p^1 \cdot (1-p)^0 = p$

y = 0: $1 \cdot (1-p)^1 = 1-p$

So, for n samples, we end up with:

$$L(w) = \prod_{i=1}^{n} p_i^{y_i} (1-p_i)^{1-y_i}$$

So, once we take logs of this,

$$\log\big(L(w)\big) = \log\left(\prod_{i=1}^{n} p_i^{y_i} (1-p_i)^{1-y_i}\right)$$

$$\ell(w) = \sum_{i=1}^{n} [y_i \log(p_i) + (1-y_i) \log(1-p_i)]$$

Minimising this, whilst also dividing by n to calculate an average loss, results in

$$\ell(w) = -\frac{1}{n}\sum_{i=1}^{n} [y_i \log(p_i) + (1-y_i) \log(1-p_i)]$$

(Within our log functions, we can add 10^{-9} to prevent a log (0) situation occurring whilst not greatly effecting our answers).

Sigmoid Function:

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Loss Function (Negative Log-Likelihood):

$$\ell(X, y, w) = -\frac{1}{n}\sum_{i=1}^{n} \text{weights}_i \cdot [y_i \log(p_i + 10^{-9}) + (1-y_i)\log(1-p_i+10^{-9})]$$

Gradient Function:

$$\nabla_w L(X, y, w) = X^T\left(weights \cdot \frac{p-y}{n}\right)$$

Where n is the number of samples

We will perform a Gradient Descent with what we have:

Using our Negative Log-Likelihood function (as defined as our Loss Function), with our parameters of:

Learning Rate = 0.1

Iterations: 2000 Epochs

initialisation: $w = 0 \in \mathbb{R}^{37}$ (A Zero Vector of 37 elements)

Our results received where the following:

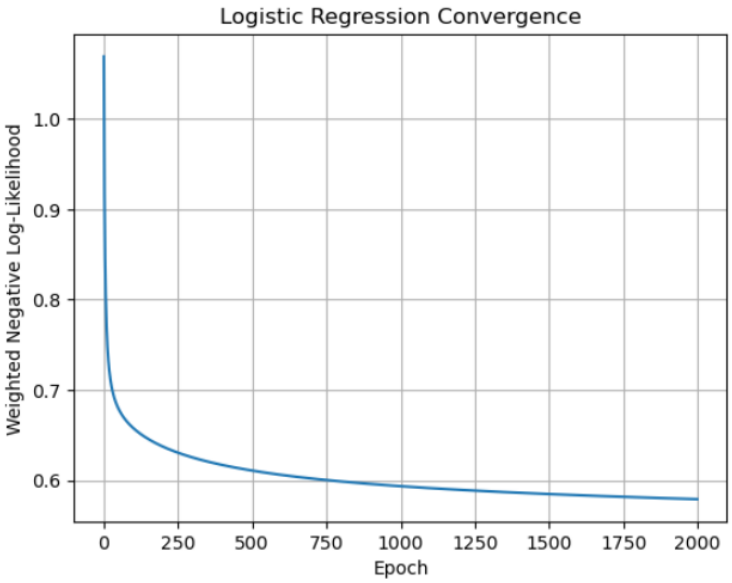
Initial Training Loss: 1.0687761462969982

Final Training Loss: 0.5793476399611275

With a Logistic Accuracy of 0.8115384615384615

This roughly tells us that our current model will reduce our loss by about 48.9%, and therefore, successfully converged over 2000 iterations (meaning that our model has found weights which help minimise the loss function, making our predictions with our model more accurate) with a Logistic Accuracy of about 81.2%, around 81% of our predictions are correct.

Plotting our Logistic Regression Convergence Graph:



This tells us that our algorithm learns fast initially between 0 and 250 epochs, then slowly plateaus more as it's converging on the optimal Weighted Negative Log-Likelihood value from 500 epochs onward. Paired with the lack of oscillations, we can comfortably say that our model is stable, and that said algorithm has effectively minimised the loss function.

However, our accuracy statement made before might be inaccurate due to how imbalanced our classes are (91% negative to 9% positive, as stated before), therefore, we must use a classification report to properly analyse the true accuracy of our model.

Classification Report:

	precision	recall	f1-score	support
0	1.00	0.79	0.88	473
1	0.32	1.00	0.49	47
accuracy			0.81	520
macro avg	0.66	0.90	0.69	520
weighted avg	0.94	0.81	0.85	520

From this classification report, we can see that we have a perfect 100% in precision when it comes to detecting negative results, but only 79% in recall, so about 21% of our actual negatives will be misclassified as positive results.

We have a perfect 100% in our recall for positive results, however, our precision on said positive results is only about 32%, which tells us that, out of 100 positive tests, 32 will be correctly positive, and 68 will be false positives.

This is present because of how we had reweighted our data to give positive values an equal stake in our algorithm, prioritising the minimisation of false negatives, this is justifiable, as a false negative may result in the patient being tested not receiving what would be lifesaving medical care, whilst a false positive only risks being inconvenienced when being re-screened for symptoms and risk.

2.3 K-Nearest Neighbours:

K-Nearest Neighbours (KNN) [2] is a non-parametric supervised learning method used as a classification (and sometimes regression) algorithm. During the testing phase, the algorithm computes the distance between the test point and all training points, selects the k closest neighbours, and will then predict the class of said points through a majority vote (If more of a particular classification exist in a given distance around the test point compared to another, then said point will be assigned the majority classification).

To calculate the distance between points, we use Euclidean Distance formula for this.

Let:

x = Test Point.

x_i = Training Point.

$d(x, x_i)$ = Distance.

As we have 37 features in our dataset, we can't just use a simple x, y, we must use a 37-Dimensional Vector instead, which changes our Distance formula into the following:

$$d(x, x_i) = \sqrt{\sum_{j=1}^{37} (x_j - x_{i,j})^2}$$

All 37 features are continuous, numeric measurements, standardised during preprocessing (Mean = 0, Variance = 1), and the Euclidean Distance is just measuring the geometric proximity between the test and training points, treating all dimensions equally.

We tested from 1 to 20 k-values and used our KNN algorithm to assess the accuracy of each k-value to find the most accurate within our given range.

```
k = 1 Accuracy: 0.9557692307692308
k = 2 Accuracy: 0.9557692307692308
k = 3 Accuracy: 0.9557692307692308
k = 4 Accuracy: 0.9557692307692308
k = 5 Accuracy: 0.9634615384615385
k = 6 Accuracy: 0.9519230769230769
k = 7 Accuracy: 0.9576923076923077
k = 8 Accuracy: 0.9519230769230769
k = 9 Accuracy: 0.9538461538461539
k = 10 Accuracy: 0.95
k = 11 Accuracy: 0.948076923076923
k = 12 Accuracy: 0.948076923076923
k = 13 Accuracy: 0.9461538461538461
k = 14 Accuracy: 0.9442307692307692
k = 15 Accuracy: 0.9461538461538461
k = 16 Accuracy: 0.9384615384615385
k = 17 Accuracy: 0.9423076923076923
k = 18 Accuracy: 0.9403846153846154
k = 19 Accuracy: 0.9384615384615385
k = 20 Accuracy: 0.9365384615384615
Best k based on accuracy: 5
Accuracy: 0.9634615384615385
```

From this, between K values 1 to 4, our accuracy remained about the same (roughly 95.6%), K = 6 accuracy was lower than our first 4 (at about 95.2%) , and beyond K = 7, we see a trend of accuracy decreasing, leaving us with our best result found at K = 5, as it gives an accuracy rating of around 96.3%, the highest accuracy value found.

Viewing our Classification Report for this:

	precision	recall	f1-score	support
0	0.97	0.99	0.98	473
1	0.89	0.68	0.77	47
accuracy			0.96	520
macro avg	0.93	0.84	0.88	520
weighted avg	0.96	0.96	0.96	520

From this classification report, we can see that we have a 97% in precision when it comes to detecting negative results, and a 99% in recall, so about 1% of our actual negatives will be misclassified as positive results.

For positives, we have an 89% in precision in detecting positive results, and a 68% in recall, so 32% of our actual positives, will be misclassified as negative results.

Unlike our Logistic Regression model, KNN has no class weighting needs. The majority voting system favours the negative class (as its unbalanced in its favour), resulting in higher overall accuracy (as denoted by our k = 5 accuracy), but a lower recall for positive cases.

This means that KNN will miss 32% of the true positives. When dealing with a potentially life-threatening illness, such high levels of false negatives may result in a significant number of deaths occurring because of delayed or no medical care.

2.4 Support Vector Machine:

Support Vector Machines (SVMs) [3] are a supervised max-margin model with associated learning algorithms that analyse data for classification and regression analysis. The goal of SVM is to maximise the margin between the two classes, the larger the margin, the better the model performs on new and unseen data.

SVM works through finding a decision boundary (known as a hyperplane) that separates the classes. In our 37-dimensional feature space, the hyperplane will be defined as:

$$w \cdot x + b = 0$$

Where:

w is the weight vector

b is the bias term (a scalar value which will shift the hyperplane)

x being a data point (one of the 37 features of a single patient)

In our situation, we used a Linear SVM for this, which will find a straight-line hyperplane to separate the classes (similar in concept to finding a y=mx+c line in 2D, but extended to 37-Dimensions in this case.)

Margin Interpretation:

The margin is the distance between the hyperplane and the nearest points of each class. These nearest points are considered our Support Vectors, as they'll define the margin boundaries.

In mathematic terms:

Margin = $\frac{2}{||w||}$

Where $||w||$ is the Euclidian Magnitude of our weight vector,

$$||w|| = \sqrt{(w_1^2 + w_2^2 + \cdots + w_{37}^2)}$$

Optimisation Objective:

SVM aims to minimise the magnitude of its weight vector, consequentially, maximising the Margin.

To maximise the defined margin, we need to make $||w||$ as small as possible, which therefore leads to:

$$\frac{(||w||)^2}{2} = \frac{1}{2}(||w||)^2$$

Where squaring also takes place to remove the magnitudes square root, leading to the largest hyperplane margin possible with any given weight.

Using LinearSVC with a C = 1 as a penalty for misclassification, and using the LinearSVC's class_weight = 'balanced' parameter to automatically balanced are severely unbalanced data, we receive the results of:

Margin = $0.6211418229534359 \approx 0.621$

$$b = -0.9522819116794136$$

Accuracy = $0.9076923076923077 \approx 90.8\%$

```
w = [-0.14432719 -0.00861277 -0.07765261  0.08360724  0.64529982  0.22300564
  0.66879321  0.13366934  0.01825521  0.37978543  0.1708675  -0.10964172
  0.08408242 -0.19541148  1.1229368  -0.70662303 -0.36075837  0.18705559
  0.05384462 -0.30117604  0.01585675 -0.12648697 -0.39818222 -0.36544926
 -0.51774269  0.19688769  0.71829316  0.12216635  0.05101362  0.1828131
  0.9688449  -2.10753722  0.44647748 -0.2693056   0.35980043  0.02514761
 -0.43347336]
```

With a classification report of:

	precision	recall	f1-score	support
0	0.99	0.91	0.95	473
1	0.49	0.87	0.63	47
accuracy			0.91	520
macro avg	0.74	0.89	0.79	520
weighted avg	0.94	0.91	0.92	520

For this classification report, whe have a 99% in precision when it comes to detecting negative results, and a 91% in recall, meaning that about 9% of our true negative results will be misclassified as positive results.

For positive classifications, only 49% in precision in detecting positive results, and 87% in its recall, so, about 13% of our true positives will be misclassified as negative.

SVM was able to handle our unbalanced data through using class weighting, and achieving a margin of about 0.621, and an overall accuracy of about 90.8%, putting it at our 2nd place in overall accuracy, in front of logistic regression, and behind KNN

However, its low precision for positives (49%) will result in many false positives, which, in practice, for roughly one positively identified patient, there was a false positive, and on a larger scale, will begin to find more false positives than true positives. This would greatly increase the wait time in real life due to the number of multiple screenings that'll take place, potentially causing a similar outcome to if said patients who were truly positive, were falsely identified as negative, increasing the medical care wait time. Even if it correctly recalls 87% of these cases, such a low precision score could make the usage of this algorithm untenable.

2.5 Comparative Analysis:

Here, I will be making the final comparison and suggestion for which model should be used within this situation, between Logistic Regression, K-Nearest Neighbours, and Support Vector Machine.

```
Final Analysis of Our Models:
---------------------------------
Logistic Regression Accuracy Score: 0.8115384615384615
Recall Score: 1.0
F1 Score: 0.4895833333333333
---------------------------------
K-Nearest-Neighbour Accuracy Score: 0.9634615384615385
Recall Score: 0.6808510638297872
F1 Score: 0.7710843373493976
---------------------------------
Support Vector Machine (SVM) Accuracy Score: 0.9076923076923077
Recall Score: 0.8723404255319149
F1 Score: 0.6307692307692307
```

Comparing each Model:

Their objective functions:

The Logistic Regression model aims to minimise the negative log-likelihood to maximize the probability of correct predictions through adjusting weights, it prioritises getting the positive classifications correct over negative classification.

KNN just measures the distance a point has with the other trained data points, and predicts the result through proximity voting, where the more that are close to an unclassified point, results in said point being classified as the majority.

SVM desires to maximise the margin through minimising the magnitude of the weight vector, it attempts to find the optimal hyperplane that splits both results. It reweights to balance the severely unbalanced data.

Putting this into the context of PCR testing for a new illness that is highly infectious, we should compare all our models on the statement of "What model is most likely to correctly identify a true positive, whilst minimising the false negatives."

For Logistic Regression, it has a recall of 100%, meaning that it theoretically should always detect a true positive, however, it isn't that precise at 32%, meaning that out of 100 people flagged positive, 68 would be a false positive, but it would always detect those 32, and be correct about it.

For K-Nearest Neighbours, it has a recall of 68%, with only 89% precision. Out of 100 people, on average, KNN may find about 32 of its truly positive individuals, classified as negative (false negative). However, for all its predicted positives, 89% are truly infected, with only a 11% being a false positive reading.

For SVM, it has an 87% recall, with a 49% in precision, out of 100 people, on average, it misses 13 of those infected, with a false positive rate at 51%.

In my judgement, it's far better to detect all that are truly infected, or had an ailment, with many false positives which can be double screened to rule out, as at most, its only inconveniencing them, and if it's not in such a supermajority (if it marked 100 as positive but only 5 are positive truly), it's a lot more useful to have, especially compared to those who will dish out false negatives.

And with that, the Logistic Regression Model is the best choice for this situation, it is better to detect all that are infected with false positives that are, at most, an inconvenience, over potentially condemning several patients to death due to them being a false negative, all parties that require medical attention can receive it.

This is even more in Logistic Regressions favour, as if what you are testing for was some infectious disease, every false negative is potentially another group of people being infected. Or even if testing for cancer, it's better to find all those who had it, and have inconvenienced people, than risk having more die due to a false negative.

Reference Page:

- [1] Wikipedia (2026). *Logistic regression*. Available at: https://en.wikipedia.org/wiki/Logistic_regression (Accessed: 21 April 2026).

- [2] Wikipedia (2026). *k-nearest neighbors algorithm*. Available at: https://en.wikipedia.org/wiki/K-nearest_neighbors_algorithm (Accessed: 23 April 2026)

- [3] Wikipedia (2026). *Support vector machine*. Available at: https://en.wikipedia.org/wiki/Support_vector_machine (Accessed: 1 May 2026)